UCF • KNIGHTS RACING • FORMULA SAE

ACTIVE AERO SOFTWARE DEVELOPMENT CONTEXT PACKAGE

Supervisory state machine • simulation architecture
Fault detection • safe-state reasoning • test design

Context snapshot 28 October 2026

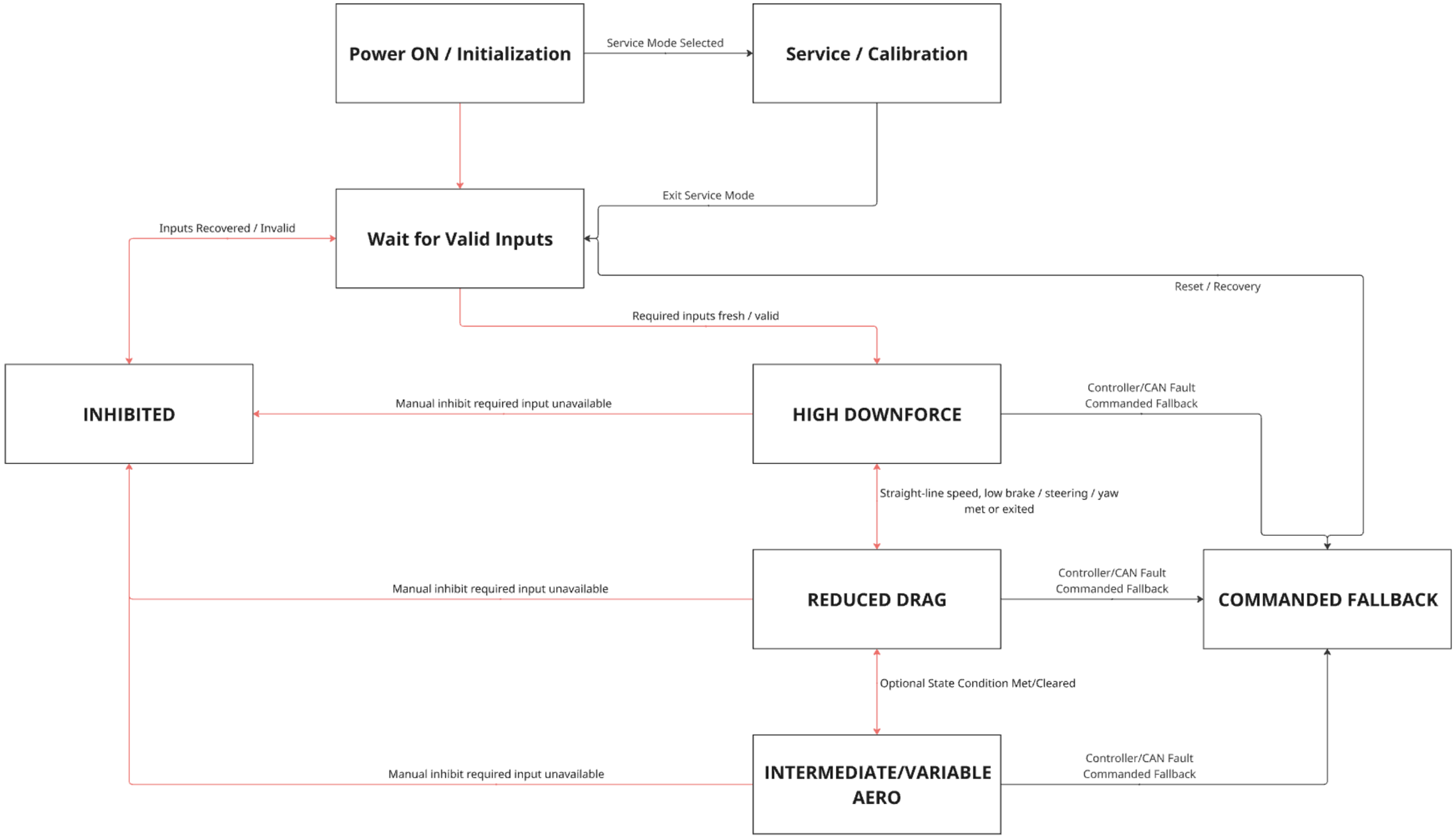
Source basis M2 working EDS (18 Oct 2026) + current COPoP

For Claude Code / software-in-the-loop development

This package separates current design context, preliminary engineering targets, and software proposals. High-downforce commands are not proof of physical element position. Total power-loss behavior remains mechanically unverified.

CURRENT COPoP STATE FLOWCHART

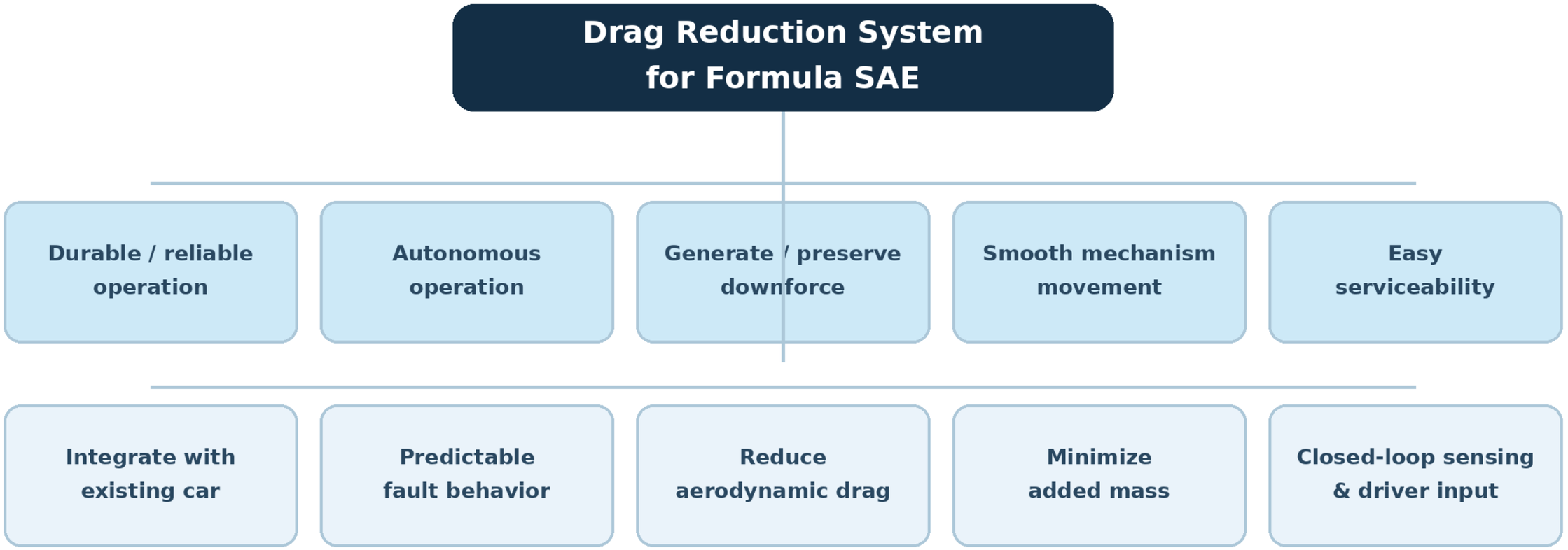
Original user-supplied chart, October 28, 2026. Labels and connectors are displayed without revision.



Simplified reconstruction based on the M2 image and project context. Not an exact screenshot of the approved decomposition.

Functional Decomposition

Simplified reconstruction of the M2 decomposition • not an export of the approved diagram



Component Decomposition

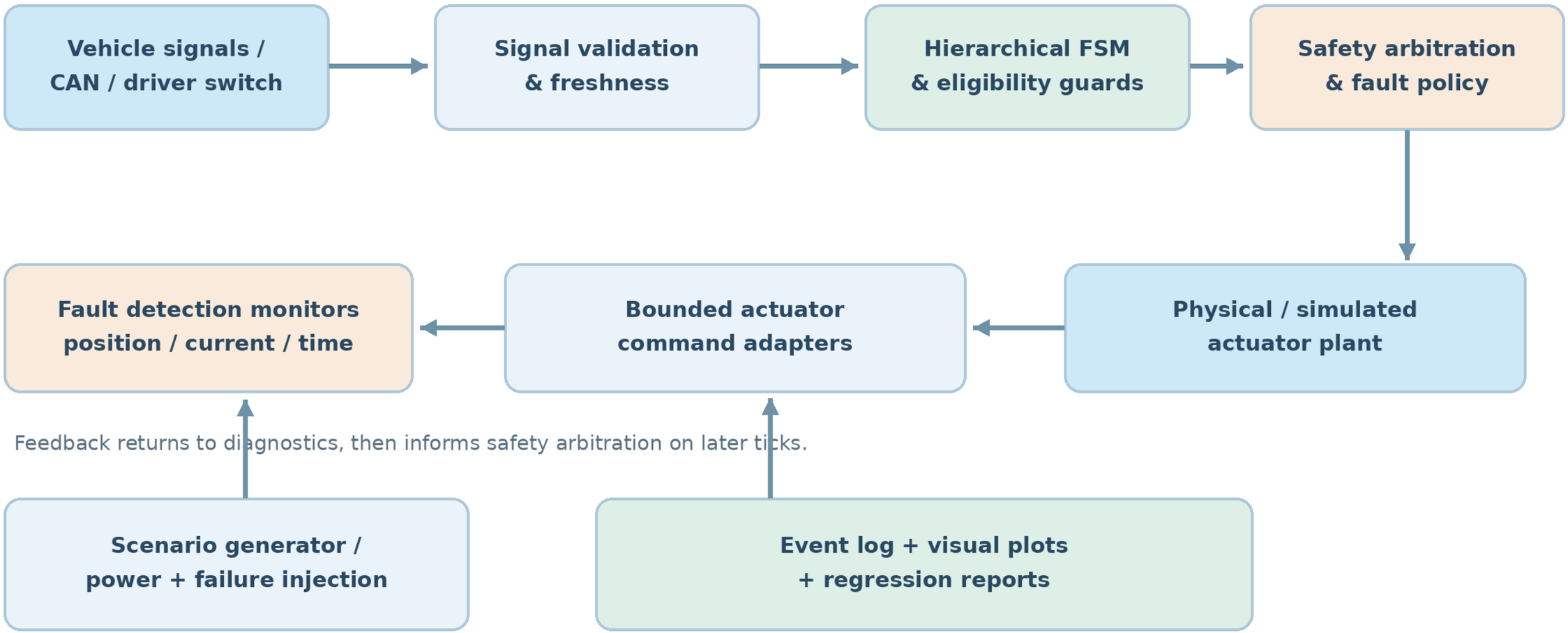
Software-oriented reconstruction from M2 • verify exact approved block decomposition separately

Active Aero System
Formula SAE

Aero Elements	Driver Interface	Actuation & Feedback	Power / Compute	Mechanical
Front left element	Tactile manual inhibit	Front servo channels	Dedicated active-aero PDM	Linkages and joints
Front right element	Driver indication	Rear geared BLDC candidate	12-to-24 V boost concept	Pivot / bushings / bearings
Rear wing element	Safety / status logging	Motor encoder & current	ESP32-S3 + CAN transceiver	Mechanical travel stops
Approved travel envelope TBD	Service controls TBD	Independent output sensor	1 Mbit/s vehicle CAN	Return-to-HDF mechanism TBD
		Optional end switches	Branch protection	Mounting / fasteners

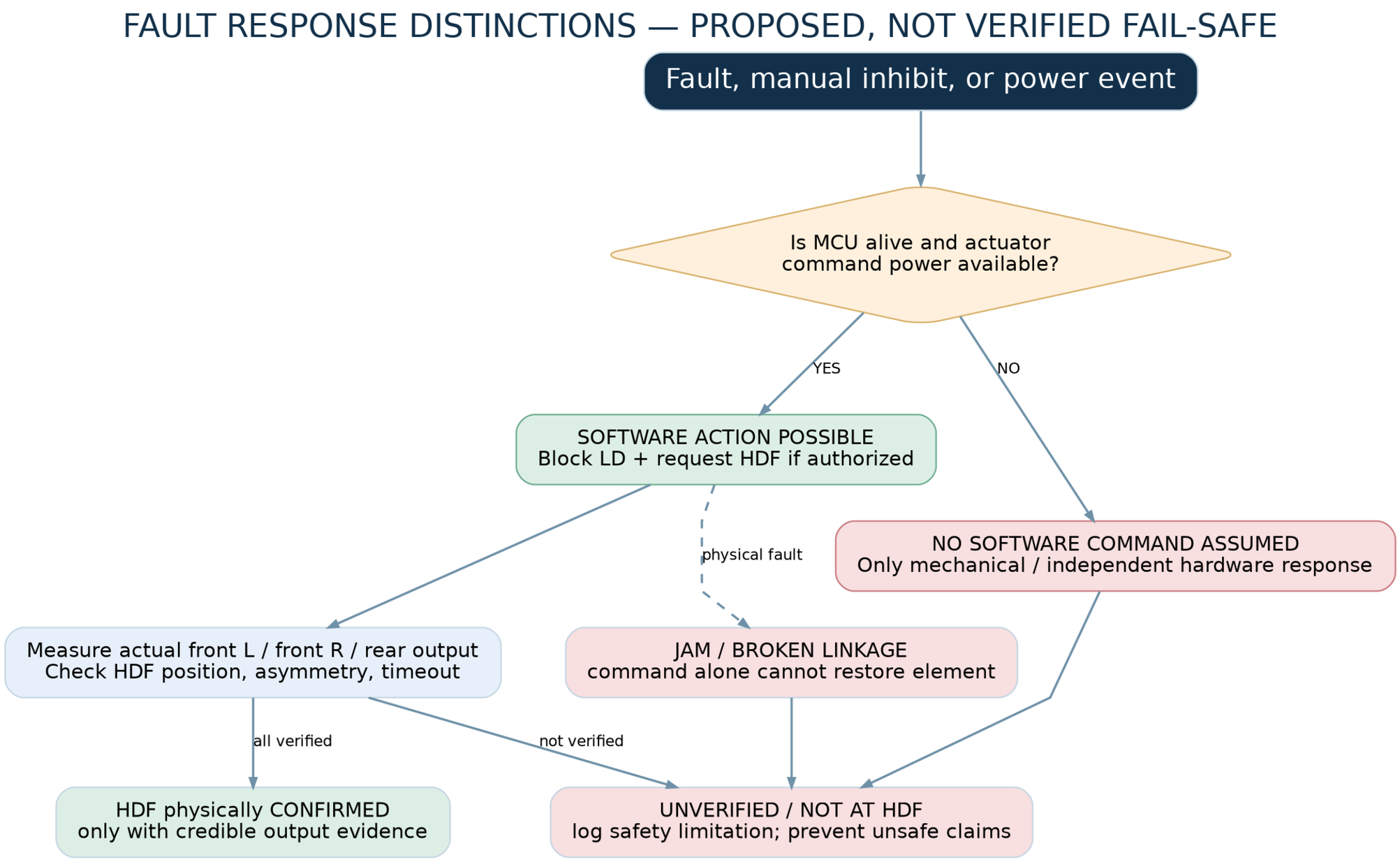
Control + Simulation Architecture

Proposed modular software design • separate decision logic from hardware I/O and physics



FAULT RESPONSE PATHS

Proposed distinction between controller-commanded fallback, lost power, and unverified physical output.



UCF Formula SAE Active Aerodynamics — Software Development Context

Snapshot date: October 28, 2026
Primary audience: Claude Code, working on supervisory state-machine logic, simulation, fault detection, and future ESP32-S3 firmware.
Authority hierarchy: (1) most recent team/advisor-approved decisions and mechanical test evidence; (2) the latest user-provided COPoP flowchart dated October 28, 2026; (3) the team's M2 *working* engineering design specification dated October 18, 2026; (4) ideas in this handoff, explicitly marked **PROPOSAL**. When a source conflicts with newer evidence, flag it and ask the team to decide; do not silently resolve safety-critical assumptions.

> **IMPORTANT:** This is a development context and simulation specification, not a certified safety case. Do not describe the vehicle as fail-safe or suitable for on-track use until mechanical, electrical, control, and Formula SAE compliance verification is completed. An HDF *command* is not proof of an HDF *physical position*.

1. Project mission and scope

Design, develop, and validate an autonomous active aerodynamic system for the Knights Racing / UCF Formula SAE vehicle. A fixed high-downforce configuration improves traction but adds drag on straights. The proposed system autonomously transitions between **high downforce (HDF)** for braking/cornering and **reduced drag (LD / DRS)** when vehicle-state inputs justify it. Intermediate/variable aero is an optional future feature only after aerodynamic validation. Driver authority, predictable behavior, fault containment, and safe operation are higher priority than lap-time benefit.

Team: four mechanical/aerodynamics engineers and two computer/electrical engineers. Advisor: Dr. Darshan Yadav. The work here is the supervisory controls and diagnostics software, using hardware-in-the-loop / software-in-the-loop simulations before vehicle integration.

Software development goals, in priority order:

1. Implement deterministic, auditable supervisory state selection with explicit safety precedence.
2. Make raw CAN/local sensor data freshness, plausibility, and validity first-class model inputs.
3. Validate actuator motion by comparing commands, internal actuator position, *independent physical output/ wing position*, current, and elapsed time.
4. Model manual inhibit and electrical/mechanical failures without assuming commanded movement occurs.
5. Provide repeatable scenario playback, injected faults, timestamped telemetry, and automated acceptance tests.
6. Keep the core controller portable to the selected MCU and the vehicle CAN/actuator interfaces.

2. Physical system and integration architecture

Aero elements and actuators

- **Front left** and **front right** are independently controlled aero elements. Front actuators are expected to be integrated position-control servos. Candidates in M2: Fashion Star RX8-C281W-M (CAN interface) and Waveshare ST3025 (serial bus). *No final actuator selection.*

- **Rear wing** is a higher-torque actuator/linkage system; one or possibly two channels remain under evaluation. M2 considers a CubeMars AK45-10 BLDC motor plus gearbox with integrated controller/encoder. *No final actuator count, mounting geometry, gear ratio, or command interface.*
- Design initially for **three independent logical channels** (`FRONT_LEFT`, `FRONT_RIGHT`, `REAR_MAIN`), but allow a fourth rear channel by configuration. Do not bake physical channel count or actuator bus assumptions into state transitions.
- A single coordinated aero configuration is the baseline. The `VARIABLE_AERO` option is feature-gated and **disabled by default** pending CFD, balance, and vehicle testing.
- Elements are intended to be *mechanically fail-closed to HDF* on total power loss, but that is a **design objective, not an established physical property**. Backdrive behavior, springs, stops, linkage friction, aero loads, and failure modes have not been demonstrated.

Sensing

- Leading approach: absolute contactless magnetic/Hall feedback, potentially at rear wing pivot/final output; options include embedded-magnet absolute encoder, packaged rotary sensor (Bosch RPS-F), magnet/shaft-based sensors, and through-bore encoder. End-stop switches are optional supplemental **binary** checks.
- Front integrated servos provide internal encoder feedback, but output-element sensing is preferable where practical. The rear wing should have **independent output position feedback**, separate from the motor encoder, particularly because a detached linkage can move the motor without moving the wing.
- Position data may be angular or linear; use typed units and calibration maps rather than assuming motor degrees = wing degrees. Define `motor_position -> expected_output_position` mapping per actuator/linkage; this may be nonlinear. Mechanical team must provide geometry and limits.
- Limit switches only confirm discrete end positions; they are not continuous position sensors.

Vehicle inputs and CAN

- Existing vehicle network is **1 Mbit/s Classical CAN**, including MoTeC M150 ECU and C125 display/logger. Candidate supervisor selected in the Oct. 18 M2 is an **ESP32-S3** using its TWAI CAN controller plus an **external CAN transceiver**. Development board is only a prototype; final custom PCB/electrical design remains pending.
- Potential inputs: vehicle speed, accelerator pedal position, brake pressure, individual wheel speeds, steering angle, yaw rate, lateral acceleration, possibly other IMU/vehicle-state signals. Some are existing CAN messages and some may be directly acquired or added to CAN. **Actual message IDs, frame layouts, scales, timing, byte order, and source validity are TBD**; do not invent a production DBC.
- M2 recommends 200 Hz (5 ms) as a **preliminary test loop**, not a final requirement. Internal acquisition, state evaluation, logs, and CAN publication may have different rates. Make all timing injectable and based on monotonic timestamps.

Electrical integration

- Vehicle low-voltage system: **12 V**, with a **dedicated active-aero PDM** and individually protected branches, separate from relying on the vehicle's existing PDM capacity.
- Current preferred architecture: one centralized **12-to-24 V boost converter** for compatible actuator channels plus appropriately regulated low-voltage supplies. Final converter sizing, actuator supply voltages, fuses, wiring, PDM, and power-loss behavior remain unverified.
- Integrated actuator controllers are preferred over custom three-phase power stages. Interfaces differ across candidates; abstract the actuator I/O behind an adapter.

- Relevant fault possibilities: lost 12 V supply, lost boost output, one actuator branch power loss, controller reset, CAN unavailable, actuator driver offline, undervoltage, overcurrent, disconnected wiring. Distinguish which components still have power during each fault.

3. Current COPoP supervisory state model (October 28 screenshot)

The **actual source screenshot** is `diagrams/copop_current_2026-10-28.png` and is included in the PDF. Treat its state names and broad relationships as the current diagram; interpret potentially ambiguous arrows explicitly instead of guessing their final semantics.

Main states shown:

- `POWER_ON_INITIALIZATION`: configure hardware, self-tests, outputs initially inhibited.
- `WAIT_FOR_VALID_INPUTS`: await fresh, valid mandatory signals and calibration/health checks.
- `HIGH_DOWNFORCE`: request default conservative HDF configuration.
- `REDUCED_DRAG`: request drag-reduction geometry only when straight-line eligibility conditions hold.
- `INTERMEDIATE_VARIABLE_AERO`: optional advanced state; disabled until approval.
- `INHIBITED`: autonomous transitions inhibited by driver switch, unavailable required inputs, or another inhibit cause. Output behavior still requires definition.
- `COMMANDED_FALLBACK`: attempt to command conservative HDF when the controller is functioning and suitable power, communication, and actuators remain available.
- `SERVICE_CALIBRATION`: controlled maintenance mode with interlocks and limited operation.

Shown transitions: initialization -> waiting; initialization -> service on explicit selection; service exit -> waiting; valid fresh inputs -> HDF; HDF <-> reduced drag when eligibility becomes true/false; reduced drag <-> optional variable aero; active states -> inhibited for driver/manual inhibit or missing required input; active states -> commanded fallback for faults; inhibit recovery -> waiting; fallback recovery/reset -> waiting. **Diagram improvement needed:** distinguish recoverable CAN/input faults, detected actuator faults, complete MCU failure, electrical power loss, and mechanical jam, because they do **not** support the same commanded fallback behavior.

Proposed hierarchy and priority (PROPOSAL — verify with team)

Represent *safety/availability mode* separately from *aerodynamic target*. Suggested top-level availability: `BOOT`, `WAIT_INPUTS`, `OPERATING`, `INHIBITED`, `FAULT_LATCHED`, `SERVICE`; nested operating mode: `HDF`, `LD`, optionally `VARIABLE`. Keep `COMMANDED_FALLBACK` as either a distinct recovery phase or a fault action with explicit progress tracking. The user screenshot's externally visible state names may be retained for logs, even if the internal structure differs.

Suggested evaluation priority per tick:

1. Hardware safety/driver override and loss of actuator power override performance modes. Hardware power loss is a **physical event**, not a software-actionable transition after controller death.
2. Controller-detected critical actuator, position, or driver faults: stop permissive behavior, latch a diagnosable fault as appropriate, and request the validated conservative action only if it can be executed.
3. Absent, stale, implausible, or invalid mandatory signals: block entry to LD; move/hold conservative as feasible; report inhibit reason.
4. Braking, steering/cornering, excessive yaw/lateral acceleration, stability concerns: HDF wins over straight-line speed/accelerator eligibility.
5. Only after all eligibility, interlocks, and actuator availability checks pass may LD be requested.

6. Variable aero may be entered only if the feature flag, state-specific constraints, and aero-balance map are independently validated.

Important nuance: A missing signal does not always require identical behavior: an optional mode-only input can inhibit LD while a critical position/actuator fault may require latching and a maintenance reset. Define a fault classification matrix, not just a single `fault: bool`.

Manual driver override (OPEN SAFETY DECISION)

The proposed tactile switch is intended to let the driver inhibit autonomous drag reduction after a problem not detected by software. The team considered physically cutting all DRS power. **Do not assume that power cutting produces HDF**: an unpowered servo, BLDC actuator, or linkage may hold LD, remain at an intermediate angle, backdrive, or jam. The safe switch topology requires mechanical and electrical verification.

Recommended interface for prototyping (PROPOSAL): model `manual_inhibit` as a highest-priority input that blocks LD, requests verified HDF when actuation is possible, and latches/logs an override indication. Separately model `actuator_power_available` and `driver_hardware_disable` to simulate a physical power cut. Consider a hardware interlock or independently powered/failsafe actuation path only after hardware review. Never claim a software switch can protect against complete MCU failure.

When manual inhibit is engaged while already in LD, distinguish `HDF_REQUESTED`, `HDF_IN_MOTION`, `HDF_CONFIRMED`, and `HDF_UNCONFIRMED`. Fault tests must cover the switch flipped during each state and during controller power loss.

4. Preliminary quantitative goals (not approved firmware constants)

These values originate in the October 18 M2 working document; they are **engineering targets or planning assumptions**, not tuned control parameters.

Source requirement	Preliminary target and caution
`R.CTL.1`	At least two aero operating states, HDF and reduced drag.
`R.CTL.2`	Select HDF for approved braking/cornering conditions; numeric triggers TBD.
`R.CTL.3`	Reduced drag intended on full-throttle straights; quantified eligibility TBD.
`R.CTL.4`	Trigger-to-initial-command/first-motion initiation time TBD.
`R.CTL.5`	Driver manual override required; switch action/topology TBD.
`R.SAF.1`	Response initiation *after fault detection* TBD; measure separately from detection.
`R.SAF.2`	Conservative HDF on fault/power loss is the **intended** state, but physical passive return is unproven.
`R.SAF.3`	No defined single fault should cause unsafe or uncontrolled motion; fault set TBD.
`R.SAF.4`	Detect defined faults within **100 ms** (HoQ proposed; fault-specific start criteria TBD).
`R.SEN.1`	Validate element/output-position feedback; accuracy, timeout, and tolerance TBD.
`R.ACT.1`	Loaded aero transition **< 300 ms** preliminary, including clearly defined start/end and load; not demonstrated.
`R.ACT.3`	Element overshoot limit TBD.
`R.PWR.1`	Total continuous active-aero draw **< 30 A from the vehicle's 12 V input** preliminary, distinct from peaks.
`R.SYS.1`	Added system mass **< 3 kg** in M2; < 2 kg internal stretch goal, not a software limit.

Source requirement	Preliminary target and caution
`R.VAL.1`	1,000 full operating cycles preliminary endurance target; conditions TBD.
`R.AER.1`	60% drag reduction proposed from CFD but matched baseline/evidence still requires verification.
`R.AER.2`	**1066.4 N at 128.7 km/h** is a reported *structural load case*, **not** an achieved downforce target.

No authoritative values available yet for speed thresholds, throttle/brake/steering/yaw limits, output position calibration, allowable error, current and timeout thresholds, motion deadlines by channel, hysteresis widths, debounce, dwell, CAN timeout by signal, serial/CAN actuator protocols, or safety reset conditions. Place these in a config schema using explicit `TBD`/unset sentinel values and refuse hardware actuation when required safety parameters are missing. For software demonstrations, a separate `simulation_example_only.yaml` may hold conspicuously synthetic values; never copy simulation-only thresholds into firmware defaults.

5. Recommended software subsystem boundaries (PROPOSAL)

Design for deterministic unit testing and a future embedded port. Keep decisions pure, time injected, and I/O adapters replaceable. Prefer Python 3.11+ for initial simulator / state-machine proofs, with stable data structures and tests; later port the core to ESP-IDF C/C++ if chosen. Do not couple decision logic directly to a GUI, CAN reader, ESP-IDF driver, or wall-clock API.

Module	Responsibility	Output
`signals`	Decode only from supplied DBC/interface definition; normalize units; attach receive timestamps and source status	`VehicleSignal<T>` and `VehicleSnapshot`
`validity`	freshness, never-received, range, invalid flags, cross-signal plausibility, timeout policy	`VALID/STALE/IMPLAUSIBLE/MISSING/SOURCE_INVALID` with reasons
`supervisor`	priority rules, state hierarchy, transitions, hysteresis, debounce, dwell, feature gating	requested aero mode + transition reason
`faults`	per-channel monitors, latching, severity, fault timers, missing data behavior	`FaultEvent`, active/latched fault set
`safety`	decide inhibit vs fallback vs disable, separate physical feasibility from request	safety action with achieved/unverified status
`actuators`	per-channel motion limits, motor/output mapping, transport adapter, acknowledgements	bounded commands and actuator statuses
`telemetry`	timestamped typed traces, reasons, raw and validated signals, fault detector outputs	CSV/JSONL and later CAN logs
`sim`	deterministic vehicle traces, actuator dynamics, command acknowledgements, failure injection	reproducible scenario executions
`tests`	state table, invariants, property tests, fault-injection, replay regression	machine-readable pass/fail evidence

Data flow (one tick): `raw messages -> timestamped normalized signals -> signal validation -> fault monitors -> safety arbitration -> aero eligibility/state decision -> bounded actuator requests -> simulated/actual feedback -> timestamped logs`. Feedback faults can be identified on subsequent ticks or using a defined ordered feedback pipeline; avoid accidental one-cycle races and make tick ordering explicit.

6. Canonical data contracts (PROPOSAL)

Use explicit units and monotonic integer timestamps (`time_ms` or `time_us`) rather than floating wall-clock time. Use `None` for not received, not zero. Vehicle speed and wheel speeds in m/s, brake pressure in bar or Pa (declare one), steering angle in rad, yaw rate in rad/s, lateral acceleration in m/s^2, normalized accelerator pedal in [0,1]. Use an actuator-specific physical output angle/position map in calibrated SI units. All diagnostics must maintain evidence that output sensors have been calibrated.

```
SignalSample:
  name, value, units, timestamp_ms, source_id, source_valid, raw_value?, sequence?
ValidatedSignal:
  value?, status {NEVER_RECEIVED,FRESH_VALID,FRESH_IMPLAUSIBLE,STALE,SOURCE_INVALID}, age_ms, reason
VehicleSnapshot:
  monotonic_time_ms; validated speed, throttle, brake, steering, yaw, lat_accel, wheel speeds
SafetyInputs:
  manual_inhibit; service_request; vehicle_power_good; actuator_supply_good;
  hardware_disable; controller_watchdog_healthy (sim-only when controller is dead)
ActuatorChannel:
  channel_id; command_position; motor_reported_position?; measured_output_position?;
  current_A?; endstop_hdf?; endstop_ld?; supply_good; comms_good;
  last_command_time_ms; last_feedback_time_ms; movement_started_time_ms?
FaultEvent:
  code; affected_channel?; first_observed_ms; detected_ms; severity; latched;
  evidence; response_action; response_started_ms?; cleared_ms?
SupervisorResult:
  high_level_state; aero_target; per_channel_target; output_confirmed;
  reason; inhibit_reasons[]; active_faults[]; transition_id; timestamp_ms
```

`output_confirmed` must be derived from **actual element measurements** (and calibrated switch indications as supplementary evidence), not command issue, motor encoder alone, or delay expiration. A missing sensor should yield `UNCONFIRMED`, not `HDF_CONFIRMED`.

7. State eligibility and transition safeguards (PROPOSAL)

- Boot: no automatic LD. Initialize peripherals, configuration CRC/version, input validity, actuator availability, initial position/limits, and startup fault flags. Never assume wing position at boot.
- Wait: require every designated **mandatory** signal to become fresh, valid, and plausible; define a configurable stabilization window. No motion unless a designated, independently evaluated safe homing/return action exists.
- HDF: conservative target when operating; HDF may be *requested* before physical confirmation. Record confirmed vs unconfirmed.
- LD entry: require valid inputs, sufficient speed/throttle/straight-line conditions, no braking/cornering/stability demand, no inhibit, no relevant fault, and compatible element availability. Require a configurable debounce and appropriate hysteresis.
- LD exit: any higher-priority safety condition promptly wins; do not delay emergency HDF request to satisfy a minimum LD dwell. Dwell may prevent noncritical thrashing, **never** block a safety exit.
- Variable aero: disabled unless approved map and test evidence are loaded.
- Inhibit: block LD; specify whether HDF request/hold is allowed. If driver switch physically removes supply, software must report hardware action as unverified and cannot guarantee movement.
- Fallback: trigger on controller-detectable critical faults when controller runs, with available command path. Latch according to fault severity. Keep physical return verification separate.

- Recovery: clear transient CAN/input recovery only after stability checks. Serious actuator, sensor, or mechanical faults should require a deliberate reset/service procedure; an automatically clearing fault can mask an intermittent mechanical failure.
- Service: require stationary/safe conditions or externally powered test rig; prevent accidental driving with service-calibration outputs enabled.

Deterministic tick pseudocode

```
function tick(now_ms, raw_samples, hardware_status, actuator_feedback):
    signals = validate_and_age(raw_samples, now_ms)
    actuator_health = inspect_position_current_timing(actuator_feedback, now_ms)
    faults = update_fault_timers_and_latches(signals, actuator_health, hardware_status)

    if hardware_status.controller_dead:              # simulator/hardware observer only
        record_physical_state_without_software_command()
    elif hardware_status.hardware_disable or not hardware_status.actuator_supply_good:
        prohibit_new_actuator_motion()
        mark_physical_position_unknown_unless_independently_measured()
    elif hardware_status.manual_inhibit:
        inhibit_LD(); request_HDF_if_mechanically_and_electrically_permitted()
    elif faults.require_latched_fallback:
        request_HDF_if_permitted(); record_fault_latched()
    elif not signals.mandatory_fresh_and_valid:
        inhibit_LD(); request_conservative_policy_if_permitted()
    elif braking_or_cornering_or_stability_demand(signals):
        request_HDF()
    elif LD_eligibility_with_debounce_and_hysteresis(signals) and actuators_healthy:
        request_LD()
    else:
        request_HDF()

    bound_and_issue_commands_if_allowed()
    log_requested_vs_measured_position_and_transition_reasons()
```

This pseudocode illustrates **priority**, not final watchdog, power topology, position return capability, or actual code implementation.

8. Diagnostic monitors and physical-failure interpretation

1. **CAN silence / stale signals:** track `last_received_ms` per message/signal. Different update rates require independent deadlines. Data can be in-range but stale. Distinguish total bus down vs a single signal missing; for a simulated dead MCU, no software action runs.
2. **Signal implausibility:** numeric/range checks, NaN, impossible jumps/rates, cross-check vehicle speed vs wheel speeds where justified; report *which* condition failed and avoid false confidence from correlated sources.
3. **Commanded-output tracking:** use per-channel `abs(expected_output - measured_output)` in physical units, only after motion-start/grace periods appropriate to expected dynamics. Detect failure to start, lag, and failure to settle. Use separate tolerances and timers for each actuator/state.
4. **Motor vs output sensor disagreement:** map motor encoder through linkage kinematics and compare against independent output sensor; persistent divergence suggests disconnected linkage, slipping coupling, or defective sensor. Fault code should indicate uncertain cause rather than declaring a jam without evidence.
5. **Stall/jam:** elevated current + low motion or stalled velocity may indicate jam. Do not assume current is available from all candidate controllers; missing current data is `UNAVAILABLE`, not zero. Consider current-sense delay/filtering and startup peaks.

6. **Travel/end stops:** commanded position past calibrated envelope, both end switches active, end switch active at impossible continuous-position reading, or HDF switch not triggered at expected HDF endpoint.
7. **Front/rear disagreement:** incompatible element configurations and left/right asymmetry can change balance; enter a validated containment strategy rather than commanding remaining elements into an unverified asymmetric arrangement.
8. **Power/communications faults:** branch voltage and actuator bus loss must be separated from CAN input loss and 12-to-24 V converter loss.
9. **Sensor drift or frozen readings:** repeated constant sensor despite motor movement, implausible jump, invalid checksum/status, disconnected sensor, independent feedback mismatch.
10. **Unknown physical behavior after no-power:** if the physical plant model lacks measured spring/backdrive parameters, mark output as `UNVERIFIED` and fail the safety acceptance criterion rather than assuming passive HDF.

Fault outcome types: `WARN` (logging only), `INHIBIT_LD`, `HDF_FALLBACK_REQUESTED`, `ACTUATOR_DISABLE`, `FAULT_LATCHED_SERVICE_REQUIRED`, `PHYSICAL_STATE_UNKNOWN`. These may overlap; avoid one giant enum if an event requires several actions. Each monitor should record detection time and the evidence it used.

Timing: the **proposed 100 ms detection** target does not mean every physically meaningful anomaly can be detected within 100 ms. Fault-specific observability, communication update periods, and detection start criteria must be established. Separate `fault_injected_at`, `first_observable_at`, `detected_at`, `response_command_at`, `movement_started_at`, and `physical_HDF_confirmed_at`.

9. Simulator concept

Build a deterministic, discrete-time software-in-the-loop simulator. Inputs can be generated mathematical traces, CSV traces from vehicle logs, or scripted scenarios. Simulator and controller must share data contracts but be separate components. Use a monotonic simulated clock and injected random seed.

Vehicle generator: speed/throttle/brake/steering/yaw/lateral acceleration time histories; optional noisy sensors, missing data, wrong units, irregular message timings, and CAN silence. Represent “driver on a straight” and “brake/corner” using profiles; do not assume unverified numerical eligibility limits are production values.

Actuator plant: three configured element channels with separately defined command-to-motion delay, maximum velocity/acceleration, travel limits, motor position, linkage ratio/curve, current model, independent output sensing, and optional endstop states. The default plant may be a **demonstration-only** simplified rate-limited first/second-order model; keep its coefficients conspicuously synthetic until bench characterization. The plant may jam, detach a linkage, stall, lose power, freeze feedback, or report a false motor position. If no-power backdrive properties are unknown, output position is unverified.

Power plant: supply available flags per branch; converter dropout; controller vs actuator supply can fail independently; driver hardware-disable switch. Distinguish a total supervisor reset from an actuator communication timeout.

Logging/plots: state timeline, HDF/LD request vs **measured** left/right/rear element positions, motor-vs-wing feedback disagreement, current, signal ages/validity, fault latch and time-to-detection, manual override, and physical-state confirmation.

Scenario file: YAML/JSON with `scenario_id`, `seed`, `duration_ms`, `tick_ms`, trace generators, fault injections (`type, start_ms, channel, magnitude, duration`), expected state/fault outcomes, and numeric tolerances. Persist machine-readable result and a short human report.

10. Minimum fault-injection / transition test matrix

Create executable tests for all cases below. Specific timings and actuator expected positions are TBD and must be parameterized. Full matrix is in `SIMULATION_SCENARIOS.csv`.

- Boot with no CAN messages -> WAIT; no LD or unverified motion.
- All required signals become fresh/valid -> enter HDF, not LD without eligibility.
- Qualified straight, consistent speed/throttle -> eligible LD only after debounce.
- Brake pressure becomes significant during LD -> HDF request has priority.
- Steering/yaw threshold exceeded -> HDF request, even with high throttle.
- Speed near threshold with noise -> no mode chatter (entry/exit hysteresis).
- Single required signal stops updating while retaining plausible numeric value -> stale/inhibit.
- Partial CAN outage vs complete CAN outage -> distinct fault/inhibit reasons.
- Vehicle-speed jump/NaN/signal invalid status -> invalidity handled, not treated as zero.
- Manual inhibit while HDF, while LD, and mid-transition -> no further permissive request; assess HDF confirmation.
- Controller execution stops / watchdog fires -> **no software fallback command from dead MCU**; report only physical/hardware response.
- Supply cut in LD -> do not assume HDF, unless a validated passive return plant is configured.
- Rear motor reports motion but independent wing sensor remains fixed -> linkage/sensor disagreement.
- Jammed rear linkage with high current and no motion -> detect if observable; record safety consequence and no false HDF confirmation.
- Sensor frozen, disconnected, delayed, or out-of-range -> explicit sensor faults.
- One front channel fails while another responds -> prohibit unvalidated asymmetric motion and report balance issue.
- Limit-switch disagreement, both end switches active, or missed switch -> fault.
- Actuator exceeds allowable position or motion timeout -> bounded command and fault.
- Fault is removed -> transient recovery vs latched-service reset conditions.
- Variable aero feature disabled -> never enter variable state.
- Service entry/exit -> controlled transition and no inadvertent driving mode.
- Logged data replay is deterministic and produces identical state/fault histories.

Core invariants: no LD when manual inhibit, critical fault, required input invalid, or stability demand; no `HDF_CONFIRMED` without independent physical evidence; state changes include reason and timestamp; no unbounded commands; power-loss simulation never fabricates motion; latches only clear under defined recovery rules; no output depends on uninitialized signals.

11. Proposed repository layout

```
active-aero/
  CLAUDE.md                          # THIS CONTEXT at repository root
  docs/
    Active_Aero_Software_Context.pdf
    diagrams/                        # source COPoP + reconstructed diagrams
    decisions.md                     # decisions, owners, approval evidence
    can_interface.md                 # message IDs/TBDs; eventually .dbc
```

```
    safety_case.md                    # FMEA, fault-to-response mapping, proof
  config/
    required_signals.yaml             # timeout and range placeholders
    actuator_channels.yaml           # channel mapping, physical envelopes
    controls_thresholds.yaml         # no production default values until approved
    simulation_example_only.yaml     # synthetic values, clearly labeled
  src/active_aero/
    models.py
    signal_validation.py
    state_machine.py
    fault_detection.py
    safety_manager.py
    actuator_interface.py
    telemetry.py
  sim/
    simulator.py
    vehicle_profiles.py
    actuator_plant.py
    power_faults.py
    scenarios/
  tests/
    test_priority.py
    test_transitions.py
    test_signal_validation.py
    test_fault_detection.py
    test_power_loss.py
    test_invariants.py
  scripts/
    run_scenario.py
    plot_scenario.py
  pyproject.toml
  README.md
```

Keep all transport and hardware-specific code behind interfaces. Use type hints, dataclasses/enums, pytest, and a static type checker if practical. Start with minimal dependencies. Never write simulated safeguards as proven physical safety systems.

12. Implementation sequence requested from Claude Code

Phase 1 — Model, not motors: create package, typed contracts, state/hierarchy definition, priority rule table, and unit tests. Run entirely offline with synthetic vehicle signals. Include current COPoP mapping in docs.

Phase 2 — Signal handling: implement timestamps, freshness, range/status validation, per-signal required/optional classification, and deterministic CAN-message replay using *mock definitions*, not invented real vehicle IDs.

Phase 3 — Actuator simulation: implement independent front-left/front-right/rear plant, command/motor/output sensors, bounded physical motion, endstops, and power/communication models. Show requested vs physical positions.

Phase 4 — Fault engine: implement independent monitors and fault records with timing, latching, severity, cause/evidence, and response path. Run jam, sensor disagreement, CAN loss, switch, and power-fault tests.

Phase 5 — Trace visualization and regression: render state/fault/position/current timelines, auto-generate pass/fail summaries, and support CSV log playback with repeatable tests.

Phase 6 — Firmware interface design: only after tests and hardware selections, define ESP32-S3 TWAI transceiver integration, actuator transport, watchdog, task/ISR scheduling, data logging, configuration storage, and measured loop execution. Keep safety validation separate from the existence of firmware code.

Do not skip to active motor control until Phase 1–4 tests run successfully. As needed, ask for actual CAN logs/DBC, actuator datasheets, linkage geometry, calibration values, mechanical fail-return evidence, and manual switch circuit design.

13. Open decisions Claude must not silently choose

1. Exact meaning and wiring of driver override: electrical power cut, hardware inhibit, dual-route, or software input; whether any independent return mechanism exists.
2. Passive HDF behavior when all actuator power is off; backdrivability, spring torque, wing aero-load direction, end stops, and failure to return under jam.
3. Safety of a *mixed* front/rear or left/right physical state, especially on a rear jam or one-sided front failure.
4. Actuator model, count (3 vs 4), bus/protocol, achievable movement time, encoder and current telemetries, and internal holding behavior.
5. Independent output position-sensor placements, calibration curve, rates, geometry, diagnostics, and final endstop mapping.
6. Actual vehicle CAN DBC/logs, signal ownership, frame update periods, actual sensor availability, signal max ages, and network loading.
7. Calibrated HDF/LD angles, speed/brake/steer/yaw/acceleration eligibility, hysteresis, dwell, filter, current, and timing thresholds.
8. Fault severity/class and which faults auto-clear vs latch/reset; what `INHIBITED` physically commands.
9. Distinct controller loss, actuator supply loss, converter loss, and CAN loss recovery behavior; whether a real independent watchdog/power switch exists.
10. Board, connector, fusing, converter, harness, and actual 12 V power distribution details; safe service/calibration interlocks.
11. Actual requirements/acceptance evidence, simulation plant parameters, 2027 competition-rule interpretation and sign-off.

14. How to collaborate with Claude Code

At repo start, put this file at root as `CLAUDE.md`; put the PDF and `diagrams/` in `docs/`. Ask Claude to **first present a proposed architecture, explicit assumptions, a prioritized task plan, and the initial state-transition truth table**. Then let it implement Phase 1 and Phase 2 with tests. Do not let it guess hardware IDs or silently make thresholds authoritative. Insist that every safety decision generates a reason code and that every output field can be simulated and tested.

Suggested task prompt

> Read CLAUDE.md and the architecture/flowchart PDF first. Implement Phase 1 of the Formula SAE active-aero supervisor as a deterministic Python package with dataclasses/enums, a hierarchical state model, an explicit priority table, signal-validity contracts, and pytest tests. Before coding, identify ambiguous COPoP transitions and safety assumptions, particularly manual inhibit, commanded HDF versus physically achieved HDF, power-loss behavior, and actuator jam. Preserve unresolved production thresholds as configuration TBDs. No hardware actuation or real CAN traffic yet. Include a runnable demo using synthetic inputs and a test report, then propose Phase 2.

15. Reference provenance and limitations

- **Current COPoP screenshot:** user-supplied image, October 28, 2026. Included unchanged in `diagrams/copop_current_2026-10-28.png` and PDF.
- **M2 WORKING.docx:** Knights Racing / UCF Formula SAE, dated October 18, 2026; Section 3.4 position sensing/fault diagnostics, Section 3.5 ESP32-S3/CAN/FSM, Section 3.6 power distribution, Section 4 requirement IDs, Appendix B functional/component decompositions. The M2 itself includes draft notes/TBDs; treat as a working source, not final approval.
- **Functional/component decomposition diagrams:** PDF includes *reconstructed, simplified vector diagrams* for software context, drawn from readable diagram labels and M2 text. They are **not** exact screenshots of the original approved diagrams; obtain approved high-resolution exports for a definitive handoff.
- **Additional software architecture/fault-path diagrams:** created specifically for this handoff as *proposals*, not diagrams taken from M2.